

1/6

```
39         self._is_clear = False # ゴールしたかどうかのフラグ
40
41     # --- ゲッター ---
42     def get_image(self): return self._image
43     def get_rect(self): return self._rect
44     def get_vel_x(self): return self._vel_x
45     def get_vel_y(self): return self._vel_y
46     def get_on_ground(self): return self._on_ground
47     def get_is_charging(self): return self._is_charging
48     def get_charge_power(self): return self._charge_power
49     def get_max_charge(self): return self._max_charge
50     def get_direction(self): return self._direction
51     def get_is_clear(self): return self._is_clear
52
53     # --- セッター ---
54     def set_vel_x(self, value): self._vel_x = value
55     def set_vel_y(self, value): self._vel_y = value
56     def set_on_ground(self, value): self._on_ground = value
57     def set_is_charging(self, value): self._is_charging = value
58     def set_charge_power(self, value): self._charge_power = value
59     def set_direction(self, value): self._direction = value
60     def set_is_clear(self, value): self._is_clear = value
61
62
63     class Platform:
64         # 足場ごとに色を変えられるように引数追加
65         def __init__(self, x, y, w, h, color):
66             self._image = pygame.Surface((w, h))
67             self._image.fill(color)
68             self._rect = self._image.get_rect(topleft=(x, y))
69
70         def get_image(self): return self._image
71         def get_rect(self): return self._rect
72
73
74     # =====
75     # 2. クラスを操作する関数
76     # =====
77
78     def update_player(player, keys, platforms, goal_block):
79         """プレイヤーの物理挙動と状態を更新する関数"""
80
81         # ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
82         if not player.get_is_clear():
83             # 1. 地面にいる時の処理
84             if player.get_on_ground():
85                 player.set_vel_x(0)
86
87                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
88                     player.set_direction(-1)
89                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
90                     player.set_direction(1)
91
92                 if keys[pygame.K_SPACE]:
93                     player.set_is_charging(True)
94                     current_power = player.get_charge_power()
95                     if current_power < player.get_max_charge():
96                         player.set_charge_power(current_power + 0.4)
```

```
97         else:
98             if player.get_is_charging():
99                 power = player.get_charge_power()
100                 direction = player.get_direction()
101
102                 player.set_vel_y(-power * 0.9 - 5)
103                 player.set_vel_x(direction * (power * 0.4 + 2))
104
105                 player.set_is_charging(False)
106                 player.set_charge_power(0)
107                 player.set_on_ground(False)
108
109     # 2. 重力の適用
110     if not player.get_on_ground():
111         player.set_vel_y(player.get_vel_y() + GRAVITY)
112
113     rect = player.get_rect()
114
115     # 座標の更新 (x軸) と壁の跳ね返し
116     rect.x += player.get_vel_x()
117     if rect.left < 0:
118         rect.left = 0
119         player.set_vel_x(player.get_vel_x() * -0.5)
120         player.set_direction(1)
121     if rect.right > WIDTH:
122         rect.right = WIDTH
123         player.set_vel_x(player.get_vel_x() * -0.5)
124         player.set_direction(-1)
125
126     # 座標の更新 (y軸)
127     rect.y += int(player.get_vel_y())
128
129     # --- 3. ゴール (天井) との当たり判定 ---
130     goal_rect = goal_block.get_rect()
131     if rect.colliderect(goal_rect):
132         # 上昇中に天井の底にぶつかったらゴール
133         if player.get_vel_y() < 0:
134             rect.top = goal_rect.bottom
135             player.set_vel_y(0)
136             player.set_is_clear(True)
137
138     # --- 4. 足場との当たり判定 ---
139     player.set_on_ground(False)
140     if player.get_vel_y() > 0:
141         for plat in platforms:
142             plat_rect = plat.get_rect()
143             if rect.colliderect(plat_rect):
144                 if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
145                     rect.bottom = plat_rect.top
146                     player.set_vel_y(0)
147                     player.set_vel_x(0)
148                     player.set_on_ground(True)
149                     break
150
151
152     # =====
153     # 3. メインループ
154     # =====
```

```
155
156  ▼ def main():
157     pygame.init()
158     screen = pygame.display.set_mode((WIDTH, HEIGHT))
159     pygame.display.set_caption("Jump King - 10 Floors to Goal")
160     clock = pygame.time.Clock()
161     font = pygame.font.SysFont(None, 36)
162     large_font = pygame.font.SysFont(None, 72)
163
164     player = Player()
165
166     # --- 10層分のマップ生成 ---
167     platforms = []
168     platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREEN)) # スタート床
169
170     TOTAL_FLOORS = 10
171     # ゴール（天井）のY座標：スタート床から 10層分(800px * 10) 上へ
172     goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
173
174     # 金色の天井ブロックを生成
175     goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)
176
177     # ゴールに到達するまで足場を生成し続ける
178     platform_y = HEIGHT - 150
179     while platform_y > goal_y + 100:
180         w = random.randint(60, 130)
181         x = random.randint(0, WIDTH - w)
182         platforms.append(Platform(x, platform_y, w, 20, GREEN))
183         gap_y = random.randint(80, 130)
184         platform_y -= gap_y
185
186     camera_y = 0
187     max_height = 0
188
189     running = True
190     while running:
191         for event in pygame.event.get():
192             if event.type == pygame.QUIT:
193                 running = False
194
195         keys = pygame.key.get_pressed()
196
197         # 関数呼び出し（ゴールブロックも渡して判定させる）
198         update_player(player, keys, platforms, goal_block)
199
200         p_rect = player.get_rect()
201
202         # --- カメラ処理 ---
203         SCROLL_TOP_MARGIN = 200
204         if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
205             camera_y = p_rect.y - SCROLL_TOP_MARGIN
206
207         SCROLL_BOTTOM_MARGIN = 150
208         camera_easing_down = 0.08
209         if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
210             target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
211             camera_y += (target_camera_y - camera_y) * camera_easing_down
212
```

```
213         if camera_y > 0:
214             camera_y = 0
215
216         # --- スコア・階層計算 ---
217         current_height = (HEIGHT - 40) - p_rect.bottom
218         if current_height > max_height:
219             max_height = current_height
220
221         # 今いる階層 (800px を 1階 とする)
222         current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
223
224         # --- 描画処理 ---
225         screen.fill(BLACK)
226
227         # ゴールの描画
228         goal_rect = goal_block.get_rect()
229         goal_draw_y = goal_rect.y - camera_y
230         if -100 < goal_draw_y < HEIGHT + 100:
231             screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
232
233         # 足場の描画
234         for plat in platforms:
235             plat_rect = plat.get_rect()
236             draw_y = plat_rect.y - camera_y
237             if -50 < draw_y < HEIGHT + 50:
238                 screen.blit(plat.get_image(), (plat_rect.x, draw_y))
239
240         # プレイヤーの描画
241         player_draw_y = p_rect.y - camera_y
242         screen.blit(player.get_image(), (p_rect.x, player_draw_y))
243
244         # チャージゲージと方向指示器
245         if player.get_is_charging() and not player.get_is_clear():
246             gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
247             pygame.draw.rect(screen, RED, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
248
249         arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
250         pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
251
252         # 情報表示 (UI)
253         ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS}   Max: {max_height // 10}m", True,
254                               screen.blit(ui_text, (10, 10))
255
256         # ゴールした時の演出
257         if player.get_is_clear():
258             clear_text = large_font.render("CLEAR!!", True, GOLD)
259             text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
260             screen.blit(clear_text, text_rect)
261
262             sub_text = font.render("CONGRATULATIONS!", True, WHITE)
263             sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
264             screen.blit(sub_text, sub_rect)
265
266         pygame.display.flip()
267         clock.tick(FPS)
268
269     pygame.quit()
270     sys.exit()
```

```
271
272     if __name__ == "__main__":
273         main()
```